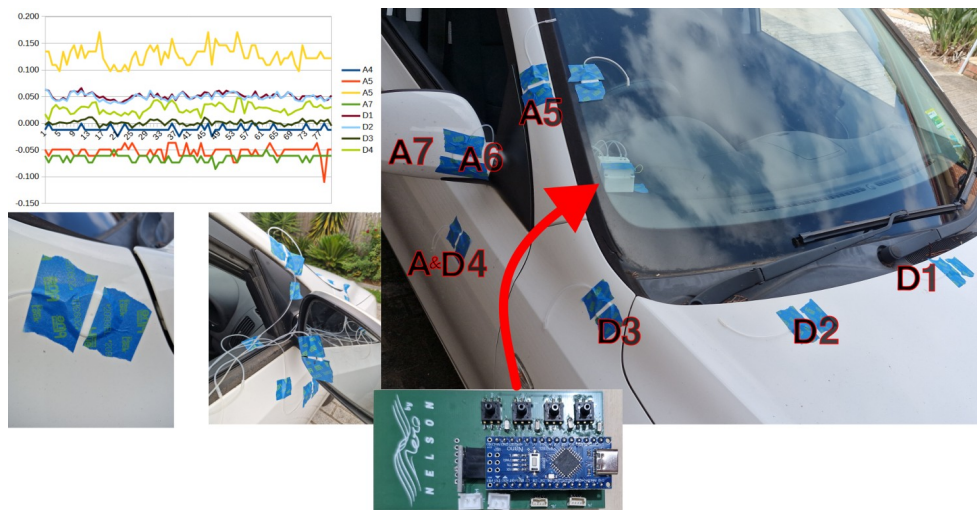




PRESSURE SENSOR KIT

USER MANUAL



Version 1.01

04/05/2026

Table of Contents

Background.....3

Basic Kit.....4

 Introduction.....4

 Setup.....4

 Use.....5

 Data Extraction.....6

Advance Kit.....7

 Introduction.....7

 Setup.....8

 Use.....8

Appendix.....10

 Calibration.....10

 The Connector Pins.....11

Background

This kit is designed to measure the air pressure over the surface of a moving car. Specifically for modified surfaces that produce aerodynamic downforce. The idea is for people that modify their own racecars to map how air moves around the car. By placing tube around the car the air pressure can be measure by the sensors provided in the kit.

The kit is intended to provide everything that is needed to start experimenting with aerodynamics of a car. It has two distinct components that allows different levels of analysis. First forms the basic functions and can function as a stand alone kit. Then the second builds off the basic kit with digital connectors which more components can be added for advanced functions.

Basic Kit

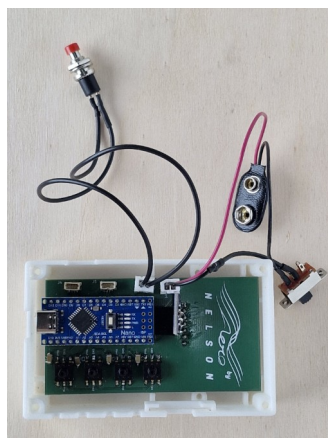
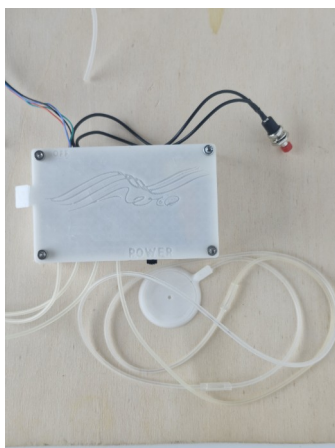
Introduction

This basic kit is designed to provide the fundamental level of air pressure measurement function. It has four factory calibrated $\pm 5\text{kpa}$ sensors connected to the 10bit analogue input of the microcontroller. A BME280 module is added for atmospheric pressure and temperature reads, helpful to provide context to a test.

Setup

The package comes with the following components:(see appendix for more detail)

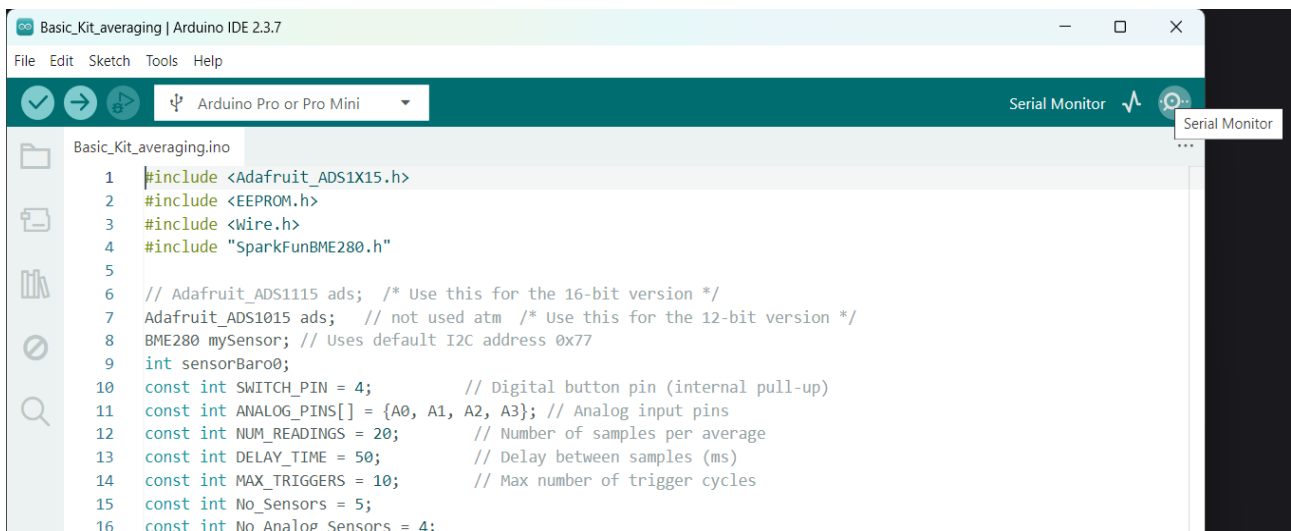
- assembled kit – excluding a 9v battery
- 10m of 1mm id silicone tube
- length of connecting tube
- assembled puck



Use

The basic kit comes with the code installed and ready to run. Connecting the device first through the usb port via the Arduino software is recommended to understand the kits functions. The kit functions exactly the same whether it is powered by the usb cable or battery.

Plugged into the computer the Arduino software the kit can interact with it through the serial monitor, top right of the UI. It opens up a window at the bottom.

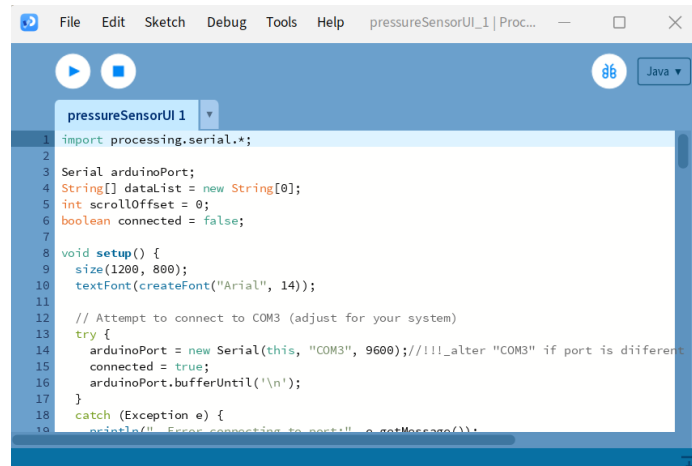


Interacting with the kit through the serial port you can write to the internal memory by typing 'w' and 'enter', pressing the **red** button does exactly the same thing. This will overwrite any data that exists from prior startups. Typing 'r' and then 'enter' will write the data to the serial and then remove that data. These are the functions the UI uses, the difference is the UI gives the opportunity the write the data to a file on you computer therefore saving the data.

Offline when it is in the vehicle and battery operated the red button is pressed to trigger the data recording. The recording takes one second with the settings displayed in the image above. Pressing the button after one second will run the data recording process again until the maximum number, 'MAX_TRIGGERS' has been met. This number can be increased. When doing the data dump if less than the 'MAX_TRIGGERS' the extra displayed sets will contain zeros.

Data Extraction

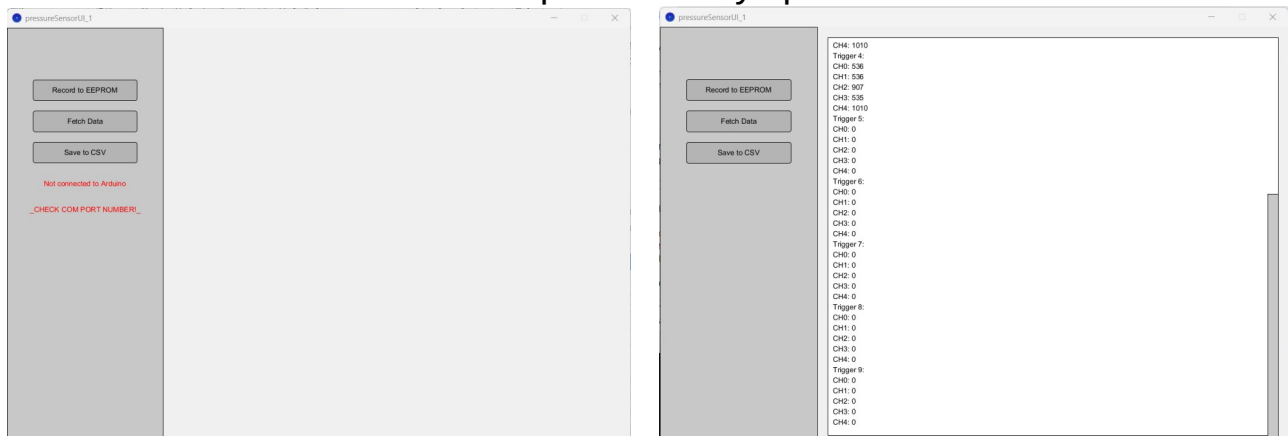
To extract and save the data requires the UI, though you could copy and paste the data out of the Arduino serial window but it doesn't work well. The UI is written in software call Processing4. After loading the code into the software it is just a matter of running it and the UI window will pop up.



```
1 import processing.serial.*;
2
3 Serial arduinoPort;
4 String[] dataList = new String[0];
5 int scrollOffset = 0;
6 boolean connected = false;
7
8 void setup() {
9   size(1200, 800);
10  textFont(createFont("Arial", 14));
11
12  // Attempt to connect to COM3 (adjust for your system)
13  try {
14    arduinoPort = new Serial(this, "COM3", 9600); //!!!_alter "COM3" if port is diiferent
15    connected = true;
16    arduinoPort.bufferUntil('\n');
17  }
18  catch (Exception e) {
19    println("Error connecting to port!! e.getMessage());
20  }
```

If the serial port is not written as the same as the one used on the Arduino, then line 14 needs to be changed(close the UI window and change it then rerun the code). (The Arduino software needs the serial window closed for Processing4 to communicate via the serial port)

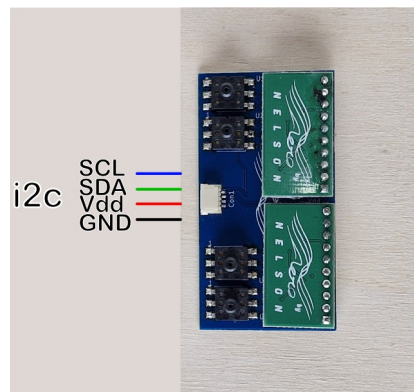
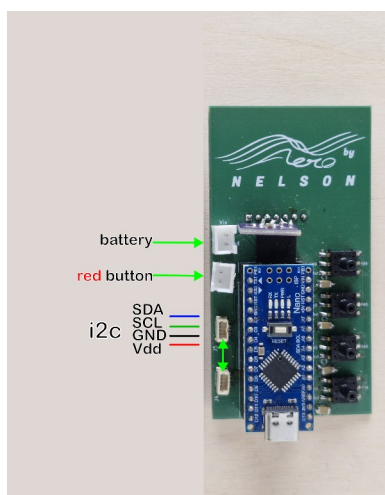
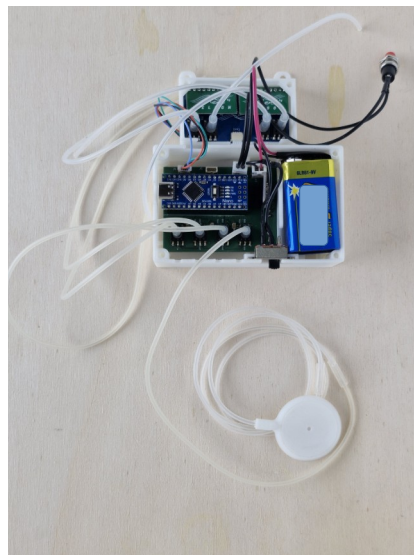
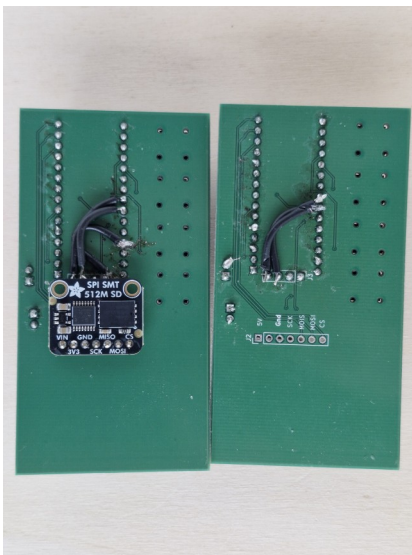
Each button on the UI writes to the kit via the same serial commands if using the Arduino software. 'Record to EEPROM' writes a 'w' to the serial, which is not necessary here as this will write over any data, only if the kit has been restarted. 'Fetch Data' will write the stored data to the UI window by sending 'r' the kit. After the data is in the window pressing the 'Save to CSV', write the data to a unique file where the pressureSensorUI_1 Processing4 code is stored. Now the CSV file can be opened in any spreadsheet.



Advance Kit

Introduction

The more advanced kit includes a memory chip/board on the back of the main board and a second array of either four or eight additional pressure sensors. This large amount of memory, 500Mb, makes it possible to increase the number of data point. That is, recording data from more sensors at a higher frequency. So the additional sensors, with the same $\pm 5\text{kPa}$ rating, are mounted on a separate board. An amplification board with a ADS1115 chip is attached produce a far more accurate reading, as such also appears more noisy.



Setup

The setup is a little different for this advanced kit, as there are two sets of code that needs to be flashed to the microcontroller. One is for the data recording and the other is for communicating to the UI to access the memory. Then if the data needs to be erased from the memory chip, an additional program needs to be loaded.

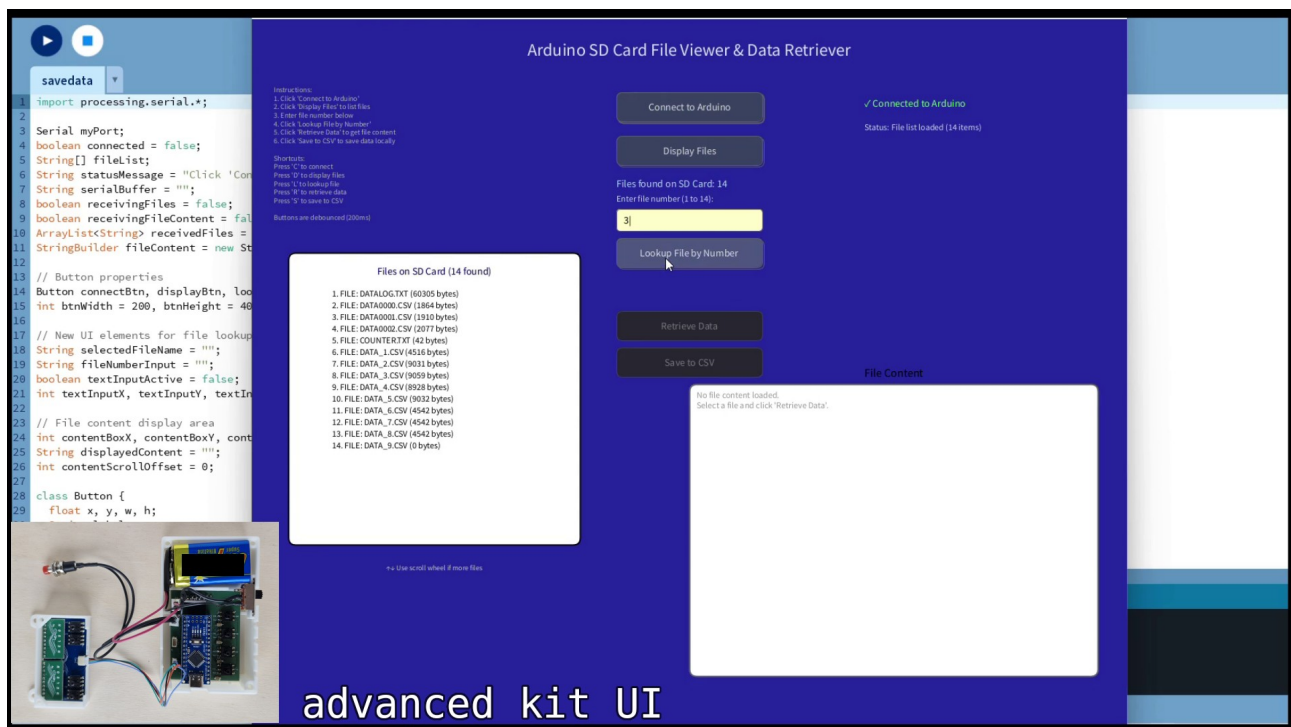
Use

The data recording code must be flashed to the microcontroller. After this it is functionally the equivalent to the basic kit. This time all the data is recorded and stored by the kit with no averaging. Each time the red button is pressed to trigger a data set and it creates a unique file. In this set of data points one is added every 100ms until the red button is pressed again to stop the recording. The frequency of the recording data points can be increased in the code if it is thought necessary from the 10Hz as seen on line 44.

```
33 // File for data logging
34 File dataFile;
35 char currentFilename[15]; // "LOG_XXX.CSV"
36
37 // Data array (max 16 values: timestamp + temp + press + 4 analog + 8 ADS1115)
38 #define DATA_ARRAY_SIZE 16
39 float dataArray[DATA_ARRAY_SIZE];
40 int dataCount = 0;
41
42 // Logging interval (100ms = 10Hz)
43 unsigned long lastLogTime = 0;
44 const unsigned long logInterval = 100;
```


Data Extraction

A separate Arduino file for interfacing with the UI is required to be flashed to the controller before the Processing4 software can extract the stored file. The UI is a crude interface but setup to follow the process in an obvious manner. Connecting to the Arduino is more straight forward than the basic kit, where the software will detect the serial port and connect to the kit by pressing the top button. It will give feedback to whether this was successful or not. If successful the following button can be pressed to retrieve the file names that will be displayed. The file that you want to extract needs its indexed number typed into the dialogue box and then the next button is pressed to display the data to be transferred to the computer. If the displayed data isn't from the right file, reenter the alternative file number in the dialogue box. The save button will transfer the data to the computer with the same name and stored where the Processing4 code is situated. Video of the process is available on the Hackaday page.

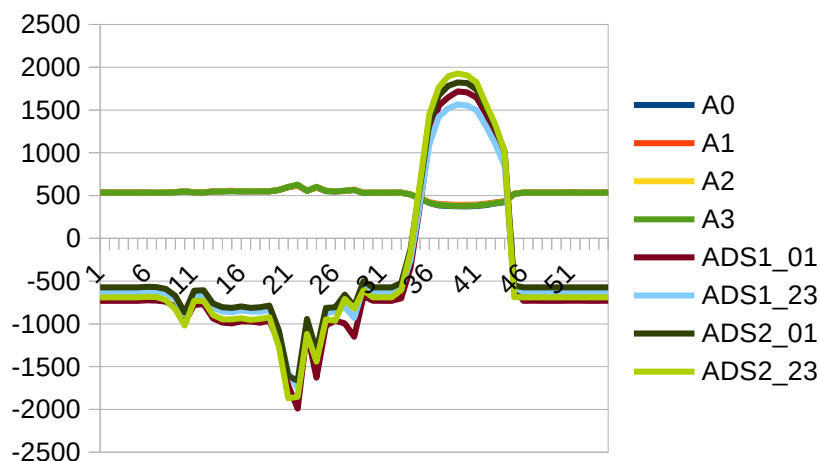


Appendix

Calibration

When an experiment is going to happen the sensors need to be zero. Thought the main board sensors are factory calibrated they will drift on the supply voltage, seen within a difference when the kit is powered by the battery or usb. Therefore it is a good idea to record a data set whilst everything is static. Then this data can be used to calibrate the rest of the sensors with a reference offset.

An example of a raw data set for calibration is below. All other the silicone tube lines are connected to a common inlet with pressure changed by applying both positive and negative by the mouth. It is apparent that the digital sensors are negative, more responsive and have a large offset.



The main board sensors use the following equation to convert the analogue input to pressure. The E5 is the spreadsheet cell and it is divided by the analogue bits to get the input voltage. The rest of the equation is give by the data sheet, 2.5 is the voltage at zero gauge pressure, 5 is the supply voltage and -6.25 is a constant.

$$=(E5/1024)*(2.5*5)-6.25$$

As the non factory calibrated sensors can drift over a number of parameters the following equation I have used may need to be altered by the user. It is highly recommended that the use characterise the sensors by using a common pressure test. Not necessarily every time it is used but at least once before the first use.

$$=(-L5-750)/750$$

The Connector Pins

The i2c connectors have a specific orientation because the horizontal and vertical version I bought change orientations.

